

# **SCENARIO-BASED MICROSERVICES INTERVIEW QUESTIONS**

Real-World Problem Solving for Senior Engineers

by Jaspreet Khera

## 1. Payment succeeds but Order fails - Distributed Rollback

Scenario: During a flash sale, PaymentService completes successfully but InventoryService fails to reserve stock. Customers are charged, but no order is placed.

Thought Process: The interviewer wants to see if you understand distributed consistency and how to recover from partial failures.

Solution Design: Use the Saga pattern with event-driven choreography. Payment emits a 'PaymentSuccess' event. If Inventory fails, an 'InventoryFailed' event triggers a compensating transaction to refund the payment. Use Kafka for event propagation and ensure idempotency in listeners.

```
@SagaOrchestration(name = 'orderSaga') public void handlePaymentSuccess() { //  
    trigger refund if inventory fails }
```

*Interview Takeaway: Ensure eventual consistency and reliability across distributed services.*

## 2. Sudden Traffic Surge - Service Crashes

Scenario: During high traffic, the checkout flow collapses as dependent services become overloaded.

Thought Process: They are testing your knowledge of scaling, throttling, and resilience.

Solution Design: Add rate limiting at API Gateway (Spring Cloud Gateway or NGINX), use Bulkhead and Circuit Breaker patterns via Resilience4j. Enable Horizontal Pod Autoscaling in Kubernetes to handle load spikes.

```
@CircuitBreaker(name = 'inventoryService', fallbackMethod = 'fallbackInventory')
```

*Interview Takeaway: Show proactive load management with resilience and auto-scaling.*

### 3. Cache-DB Inconsistency - Stale Data Issue

Scenario: ProductService updates DB, but cache still shows outdated data, leading to wrong pricing.

Thought Process: Tests your data synchronization and cache invalidation strategy.

Solution Design: Adopt cache-aside or write-through strategy. On data update, evict or refresh cache entries. Use Kafka event to notify other microservices of updates.

```
@CacheEvict(value='products', key='#id') public void updateProduct(Product p) {  
    repo.save(p); }
```

*Interview Takeaway: Demonstrate your understanding of consistency and cache coherence.*

## 4. Long-Running Requests - Slow User Response

Scenario: The checkout API takes too long because all operations (payment, invoice) are synchronous.

Thought Process: They want to assess your ability to design asynchronous flows.

Solution Design: Offload slow tasks to asynchronous message queues (Kafka/RabbitMQ). Return an acknowledgment immediately and process background jobs separately.

```
producer.send('order-events', orderData);
```

*Interview Takeaway: Design responsive systems using async communication.*

## 5. API Contract Changes - Client Compatibility

Scenario: Frontend breaks after backend modifies API response format.

Thought Process: Evaluates your approach to API evolution and versioning.

Solution Design: Version APIs (/v1, /v2) or use header-based negotiation. Maintain backward compatibility while gradually migrating clients.

```
@GetMapping('/api/v2/products')
```

*Interview Takeaway: Maintain stability while evolving services incrementally.*

## 6. JWT Token Expiry - Auth Failures Between Services

Scenario: Inter-service calls fail intermittently due to expired JWT tokens in long chains.

Thought Process: Checks your grasp of distributed authentication and security.

Solution Design: Use short-lived access tokens with refresh tokens. For inter-service communication, adopt OAuth2 client credentials or mTLS for internal trust.

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://auth-server
```

*Interview Takeaway: Demonstrate secure, seamless service-to-service authentication.*

## 7. Missing Correlation in Logs - Hard to Debug

Scenario: Logs from multiple microservices can't be correlated for one request.

Thought Process: Tests your observability and monitoring setup.

Solution Design: Implement distributed tracing via OpenTelemetry. Propagate traceId through HTTP headers and include it in logs using MDC (Mapped Diagnostic Context).

```
MDC.put('traceId', UUID.randomUUID().toString());
```

*Interview Takeaway: Highlight traceability and visibility across systems.*



## 8. Gradual Monolith Migration - Strangler Fig

Scenario: Need to migrate a legacy monolith to microservices without downtime.

Thought Process: Tests your strategy for safe and incremental migration.

Solution Design: Use Strangler Fig pattern ? introduce an API Gateway to reroute requests for specific modules (like Orders) to new microservices, phasing out the monolith gradually.

```
route('/orders/**').uri('lb://ORDER-SERVICE')
```

*Interview Takeaway: Show your ability to evolve architecture without disruption.*

## 9. API Gateway Latency - Performance Bottleneck

Scenario: Clients experience slow response times due to overloaded API Gateway.

Thought Process: Evaluates your debugging and optimization mindset.

Solution Design: Enable caching, async routing, and circuit breakers at the gateway. Tune thread pools and increase connection limits for optimal performance.

```
spring.cloud.gateway.httpclient.pool.max-connections=200
```

*Interview Takeaway: Show your understanding of performance tuning at the gateway layer.*

## 10. Data Duplication Across Services - Sync Problems

Scenario: Multiple microservices maintain partial user data, causing mismatches.

Thought Process: Tests your event-driven data design approach.

Solution Design: Adopt CQRS and event sourcing ? publish domain events (UserUpdated) and let other services subscribe to maintain eventual consistency.

```
eventPublisher.publish('UserUpdated', user);
```

*Interview Takeaway: Show how to maintain data ownership and consistency via events.*